

1 问题与动机

对于数组而言，其最重要的性质是随机访问：给定下标 i ，只需根据首地址和元素宽度计算目标地址，便能在 $O(1)$ 时间内读取或修改元素。然而，传统连续数组的容量在分配时已经固定。若数组之后需要增长，紧随原数组之后的地址可能已被其他对象占用，算法便不能保证原地延长原有内存区间。因此，可变长数组必须借助动态内存管理。

动态内存管理不只意味着申请一个更大的连续数组。其实现至少有两条基本路线：

1. **连续重分配。** 申请一个更大的连续数组，把旧元素复制过去，再释放旧数组；几何扩缩容属于这一路线。
2. **分块表示。** 把一个逻辑数组分散到多个动态申请的固定长度数据块中，再用索引块保存各数据块的指针；HAT、Brodnik 结构以及 Tarjan-Zwick 的新构造都采用这一路线。

无论采用哪条路线，外部看到的仍是同一个按下标排列的逻辑数组。区别只在于元素是否物理连续，以及实现如何在更新时间、索引开销、未使用槽位和重组峰值之间取舍。

本文研究的抽象数据类型支持 $\text{Get}(i)$ 、 $\text{Set}(i, x)$ 、 $\text{Grow}(x)$ 和 $\text{Shrink}()$ 。这里的“末端”或论文所说的“远端”，是指当前最大下标一侧：若数组原来包含 $a_0, a_1, \dots, a_{(N-1)}$ ， $\text{Grow}(x)$ 产生的新元素是 $a_N = x$ ， $\text{Shrink}()$ 删除的是 $a_{(N-1)}$ 。因此，末端更新不会改变任何保留元素的下标。

这一操作边界与任意位置插入、删除的动态序列有本质区别。例如，在

$[a_0, a_1, a_2, a_3]$

的位置 1 插入 x 后，序列变成

$[a_0, x, a_1, a_2, a_3]$ 。

原来的 a_1, a_2, a_3 都获得了新下标，数据结构不仅要容纳一个新元素，还要维护后续元素变化后的秩。本文的末端 Grow/Shrink 不产生这种全局秩变化，所以不能只比较大 O 数值就把两类问题视为相同。论文的单端结果也可以组合成双端可变长数组：分别用两个单端结构保存中心点左右两侧，其中一侧按相反顺序编号。双端更新仍只作用于两个物理末端，也不等于任意位置插删。

最常见的连续重分配方案是几何扩缩容。它保持最坏 $O(1)$ 的随机访问和摊还 $O(1)$ 的末端更新，却可能长期保留 $\Theta(N)$ 空槽，并在重建时让新旧两个线性大小的数组短暂共存。HAT 和 Brodnik 等分块方案把冗余降低到 $O(\sqrt{N})$ ，但必须额外维护指针和定位规则。

不过，如果仅写“某个方案使用 $O(N)$ 空间”，则会掩盖两个不同问题：数组在一次操作完成后需要长期保留多少空间，以及一次扩缩容正在执行时会短暂达到多高的空间峰值。Tarjan 和 Zwick 的关键出发点，就是分别度量这两个量。

2 模型与双空间指标

2.1 内存块模型

根据论文原文所述，其将具体实现视为一组动态申请的固定长度内存块（blocks）。数据块保存元素，索引块保存指针，辅助块保存长度或计数。若当前数组有 N 个机器字大小的元素，则数据本身至少需要 N 个机器字；除一个主索引块外，每个可访问内存块还需要被某个指针指向。这里的“重新分配”和“重建”含义不同：重新分配（**Reallocate**）是内存管理接口对单个连续块进行扩展、搬移或替换；重建（**rebuild**）是数据结构重新组织多个数据块、索引块和元素的过程。一次重建可能调用多次申请、释放或重新分配。

2.2 稳定空间与扩缩容临时空间

对两个非递减函数 $s(N)$ 和 $t(N)$ ，若一个实现满足：

稳定保存大小为 N 的数组：最多 $N+s(N)$ 空间；
执行 **Grow** 或 **Shrink** 期间：最多 $N+t(N)$ 空间，

则称其为 $(s(N), t(N))$ -实现。本文把 $s(N)$ 统一称为稳定冗余，把 $N + s(N)$ 称为稳定存储空间；把 $t(N)$ 称为扩缩容临时冗余，把 $N + t(N)$ 称为扩缩容临时空间。“稳定”表示一次操作完成后的表示，不表示这些内存永久不释放；“临时”强调 **Grow/Shrink** 的执行过程，而不是另一种独立的数据结构。

双指标的意义在于，应用可能同时保存许多可变长数组，却只在某一时刻调整其中少数数组。此时大多数数组可以保持紧凑表示，共享的工作内存只为正在重组的结构提供短暂空间。不过，这只是理论动机；实际收益仍受缓存、内存分配器、碎片和并发重组影响。

2.3 全文术语约定

为避免把不同论文的局部名称误认为不同概念，本文采用以下主要术语：

原文或局部名称	本文主要名称	使用规则
Top	索引块	介绍 Sitarski 原文时写作“Top (索引块)”
Leaf	数据块	介绍 Sitarski 原文时写作“Leaf (数据块)”
virtual superbblock	虚拟超级块	仅表示 Brodnik 对同尺度数据块的概念分组，不是实际申请的内存块
Reallocate	重新分配	指内存接口对一个块的操作
rebuild	重建	指数据结构层面的重新组织
access	访问	包括 Get 与 Set 所需的下标定位和读写
amortized time	摊还时间	针对任意操作序列的总成本，不写成“平均时间”
standard implementation	标准实现	仅在 Sections 7-9 的作者定义范围内使用
growth game	增长博弈	指时间下界所用的抽象博弈

有了操作模型和双空间指标，下面可以把旧方法放到同一坐标系中比较：精确容量方案追求最小稳定冗余却频繁复制，几何扩容容以线性冗余换取简单和快速，HAT 与 Brodnik 则把任意时刻的冗余降到平方根级。

3 三类旧方法

3.1 精确容量与几何扩容

若每次 Grow 或 Shrink 都申请恰好匹配新长度的连续数组，稳定冗余只有 $O(1)$ ，但每次更新都要复制 $\Theta(N)$ 个元素，重建时总空间接近 $2N$ 。它展示了极小稳定空间的一端。

几何扩容位于另一端。设固定扩容因子为

$$g = 1 + \alpha, \alpha > 0.$$

当容量为 N 的连续数组刚好装满时，结构申请一个容量约为

$$(1 + \alpha)N$$

的新数组，复制原来的 N 个元素，然后释放旧数组。忽略取整和触发扩容的单个新元素，重建刚结束时大约新增 αN 个空槽，下一次装满前也会发生大约 αN 次 Grow。一次重建复制 N 个元素，因此分摊到这些更新上的复制成本为

$$N/(\alpha N) = 1/\alpha$$

也就是 $\Theta(1/\alpha)$ 。若把每次 Grow 自身写入新元素的常数成本也计入，论文给出的表达式与 $(1 + \alpha)/\alpha$ 同阶。因而 α 变小会产生一组方向相反的影响：

alpha 变小

-> 每次扩容预留的空槽减少

-> 两次扩容之间可容纳的 Grow 数量减少

-> 扩容更频繁

-> 每次 Grow 的摊还复制成本增大

两个数字例子可以显示这一差别：

增长因子 g	α	刚扩容后的新增空槽	约需多少次 Grow 再次装满	分摊复制量级
2	1	N	N	约 1 次复制/操作
1.25	0.25	$0.25N$	$0.25N$	约 4 次复制/操作

这里“约 1 次”或“约 4 次”指元素复制次数的摊还量级，不表示一次操作的实际墙钟时间，也忽略了取整、分配器和元素构造成本。增长因子为 2 时，空闲槽数最多可接近当前元素数，也就是已分配容量可能接近一半未使用；增长因子为 1.25 时，刚扩容后的空闲槽约占新容量的 $0.25/1.25 = 20\%$ ，但重建明显更频繁。

扩容不能简单地在使用率刚低于 $1/g$ 时立即进行。否则数组在边界附近执行一次 Shrink 后扩容，紧接着一次 Grow 又可能重新扩容，两个昂贵重建无法由足够多的普通操作分摊。标准做法是使用滞后阈值：若当前容量为 C ，可以等到元素数降至约

$$C/g^2$$

时，才把容量缩小到约 C/g 。扩容后装载率约为 $1/g$ ，离再次扩容或扩容都保留了一段缓冲区间。这个滞后区间保证相邻重建之间发生线性数量的更新，从而维持 $O(1)$ 摊还成本。

对任意固定 $\alpha > 0$ ，几何方案在稳定状态中保留的空槽最多为 $\Theta(\alpha N) = \Theta(N)$ ，而重建时旧数组和容量约为 $(1 + \alpha)N$ 的新数组同时存在，临时冗余也是 $\Theta(N)$ 。采用后台复制可以把一次大规模重建分散到后续操作，从而获得最坏情况常数更新时间，但这不会自动消除新旧表示共存所需的线性空间。由此得到几何扩缩容的完整取舍：它用线性冗余换取简单地址计算和常数更新界，而减小增长因子只能改善常数比例，不能把冗余降为 $o(N)$ 。

线性冗余并不是常数更新时间的必要代价。Sitarski 的 HAT 改用等长数据块，只在较少的尺度变化时重建，从而把下面要讨论的冗余降到平方根级。

3.2 Sitarski 的 HAT

即便名字中包含“哈希”二字，Sitarski 的哈希数组树（hashed array tree, HAT）并不使用哈希，结构也不是一棵通常意义上的树，而是一个索引块指向若干等长数据块的两级表示。对当前长度 N ，结构维护一个通常取为 2 的幂的参数 B ，并保持

$$\sqrt{N} \leq B < 4\sqrt{N}.$$

HAT 的结构不变量如下：

- 一个长度为 B 的索引块 $\mathbf{I}$ ，其中每个已使用位置保存一个数据块指针；

- $\lceil N/B \rceil$ 或 $\lceil N/B \rceil + 1$ 个长度均为 B 的数据块;
- 前面的数据块全部填满, 只有最后一个非空数据块可以部分填充;
- 结构至多保留一个完全空的数据块, 以避免一次 **Shrink** 刚释放末块、下一次 **Grow** 又立即重新申请;
- 所有逻辑元素仍按下标顺序依次排列在这些数据块中。

因此, 对从 0 开始的下标 i , 元素位置可直接计算为

$$\text{block}(i) = \left\lfloor \frac{i}{B} \right\rfloor$$

$$\text{offset}(i) = i \bmod B.$$

访问过程先读取 $I[\text{block}(i)]$ 得到数据块指针, 再读取该块的 $\text{offset}(i)$ 位置, 只进行常数级内存访问和整数运算。若 B 是 2 的幂, 除法和取模还可分别由右移和位掩码实现, 所以 **Get** 与 **Set** 都是最坏 $O(1)$ 。

Grow 在当前尺度尚有容量时分为三种情况:

1. 最后一个数据块未满, 直接把新元素写入其下一个空位;
2. 最后一个数据块已满, 但结构已经保留一个空数据块, 把新元素写入该空块首位;
3. 最后一个数据块已满且没有预留空块, 申请一个新的长度 B 数据块, 在索引块中登记其指针, 再写入新元素。

当 $N = B^2$, 长度为 B 的索引块已经指向 B 个满数据块, 当前尺度达到容量上限。结构把参数加倍为 $2B$ 并重建。**Shrink** 的普通情况与 **Grow** 对称: 删除最后一个元素; 末端块变空后可以暂不释放, 只有在末端出现两个空块时才释放其中一个。当数组缩小到 $N = B^2/16$, 等价于当前 $B = 4\sqrt{N}$, 结构把 B 减半并重建。扩张和收缩使用不同阈值, 避免操作序列在一个边界附近来回触发重建。

稳定状态的冗余可以逐项计算:

索引块	B
最后一个部分填充数据块的空位	$< B$
至多一个预留空数据块	B
长度、计数等辅助信息	$O(1)$

因此总空间至多为 $N + 3B + O(1) = N + O(\sqrt{N})$ 。改变 B 时不必同时申请另一套包含 N 个槽的完整结构; 新数据块可以逐个申请、填充, 旧数据块也可逐个释放, 所以重建只需 $O(\sqrt{N})$ 级临时冗余。另一方面, 加倍或减半 B 会使可表示容量改变常数倍, 两次同方向重建之间至少发生 $\Omega(N)$ 次 **Grow** 或 **Shrink**, 故一次 $O(N)$ 重建可摊到这些操作上, 每次更新的摊还代价仍为 $O(1)$ 。

下面是一个自行绘制的小例子。令 $B = 4, N = 13$, 索引块有 4 个位置, 前三个数据块填满, 第四个数据块只含元素 **a12**:

```

index block I (length 4)
+-----+-----+-----+-----+
| D0 | D1 | D2 | D3 |
+---+---+---+---+
|   |   |   |   |
|   |   |   |   |
|   |   |   |   |
v     v     v     v
D0     D1     D2     D3
+-----+-----+-----+-----+
|a0|a1|a2|a3| |a4|a5|a6|a7| |a8|a9|a10|a11| |a12| | | |
+-----+-----+-----+-----+

```

例如 $i = 10$ 时, $\lfloor 10/4 \rfloor = 2$ 且 $10 \bmod 4 = 2$, 所以 **a10** 位于 **D2** 的偏移 2。这个例子也直接显示: 末块浪费 3 个槽, 但浪费始终小于 B 。

HAT 说明可变长数组并不必须浪费常数比例空间: 分块和一级索引能够把冗余降到 $O(\sqrt{N})$, 同时保留常数访问和常数摊还更新。它仍会在参数 B 改变时重建数据布局; Brodnik 等人的结构进一步改变块尺度组织, 使数据元素不必周期性整体搬移。

3.3 Brodnik 等人的结构

在Sitariski 的哈希数组树之外, Brodnik、Carlsson、Demaine、Munro 与 Sedgewick 提出了另一种 $N + O(\sqrt{N})$ 的可变长数组。论文给出两个数据布局变体: 第一种使用严格递增的数据块; 第二种把数据块大小限制为 2 的幂, 以便避免访问定位中的平方根计算。

第一种变体把元素按顺序放入容量为

1, 2, 3, ..., k

的数据块。前 k 个块的总容量为三角数

$$1 + 2 + \dots + k = k(k+1)/2.$$

因此, 足以保存 N 个元素的最小块数为

$$k = \left\lceil \frac{\sqrt{8N+1}-1}{2} \right\rceil = \Theta(\sqrt{N}).$$

除最后一个块外, 其余块都填满; 最后一个块可以部分填充。结构还可以保留一个曾经含有元素、后来因 **Shrink** 变空的下一数据块。一个大小为 $\Theta(\sqrt{N})$ 的索引块保存这些数据块的指针, 而索引块自身使用最朴素的动态数组方法扩展。数据块、尾块空位、数据块指针以及索引块的未用位置都只有 $O(\sqrt{N})$ 量级, 所以稳定总空间为 $N + O(\sqrt{N})$ 。

该布局的 **Grow** 流程是:

1. 若最后一个非空数据块尚未填满, 直接写入其下一个位置;
2. 若该块已满但已经存在一个预留空块, 把新元素写入空块首位;
3. 否则申请容量为下一整数的新数据块, 并把指针加入索引块;

4. 若索引块已满，则申请更大的索引块并复制全部数据块指针；若要求最坏 $O(1)$ 更新时间，这次指针复制需在后续操作中后台完成。

Shrink 删除最后一个元素。若末端数据块变空，可先把它作为预留空块保存；当继续收缩而不再需要它时再释放，并从索引块删除相应指针。这里移动的主要是索引指针，已经装入数据块的元素不会因为全局尺度变化而被周期性整体搬迁。这是它与 HAT 的重要区别。

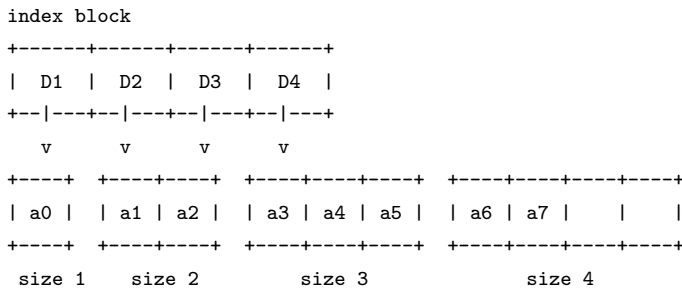
第一种变体的访问定位来自反解三角数。为避免 0/1 下标混淆，令 $p = i + 1$ 表示元素的 1-based 位置，令 j 表示 1-based 数据块编号，则 j 是满足 $j(j+1)/2 \geq p$ 的最小整数，即

$$j = \left\lceil \frac{\sqrt{8p+1} - 1}{2} \right\rceil.$$

该元素在第 j 个数据块中的 0-based 偏移为

$$offset = p - j(j-1)/2 - 1.$$

例如 $N = 8$ 时，块容量依次为 1, 2, 3, 4，第四块只使用前两个位置。访问 $i = 6$ ，即 $p = 7$ ，可得 $j = 4$ 、 $offset = 0$ ，所以该元素位于第四块首位。



直接计算上式需要整数平方根。为避免依赖该操作，第二种变体只使用大小为 2 的幂的数据块，并把它们组织成虚拟超级块 (virtual superblocks)：第 q 个虚拟超级块由 $2^{\lceil q/2 \rceil}$ 个、每个容量为 $2^{\lceil q/2 \rceil}$ 的数据块组成。虚拟超级块只是对相同尺度数据块的逻辑分组，并没有对应的整块内存申请。由元素下标的最高有效位和若干移位即可确定虚拟超级块、其中的数据块以及块内偏移。正文不需要复现其全部位级公式，但必须保留这一逻辑：第一种变体通过反三角数定位，第二种变体通过幂次分组把定位转化为 word-RAM 字操作；在相应机器运算假设下，两者都支持最坏 $O(1)$ 访问。

HAT 与 Brodник 结构的差异可以概括如下：

方面	HAT	Brodник 等人的结构
数据块大小	全部为当前参数 B	递增大小，或按 2 的幂组织
下标定位	商和余数	反三角数，或最高有效位与移位
数据元素重建	B 改变时需要重建数据布局	不需要周期性整体搬移数据元素
索引处理	改变 B 时一并重建	索引块可独立扩展并后台复制
空间	$N + O(\sqrt{N})$	$N + O(\sqrt{N})$
更新界	$O(1)$ 摊还；可进一步去摊还化	在分配假设和后台索引复制下最坏 $O(1)$

两种方案从不同结构出发，却共同达到 $N + O(\sqrt{N})$ 。这使问题自然转向下界：平方根是否只是两个设计恰好得到的结果，还是单一峰值空间指标下不可突破的屏障？Brodnik 等人的论证回答了后一个问题。

4 为什么旧下界仍允许新结果

有别于新下界将稳定状态和重建瞬间严格区分，旧下界把稳定状态和重建瞬间统一计入“任意时刻空间”，从而使得旧下界需在所有时刻严格保持 $N + O(\sqrt{N})$ 。Tarjan 和 Zwick 观察到， $N + \Omega(\sqrt{N})$ 只要求某个历史时刻出现该冗余，而这个时刻可能恰好是新块刚被申请、旧表示尚未释放的扩缩容过程。因而，只要接受扩缩容期间更高的临时峰值，一个结构仍可能在操作结束后只保留远少于 $\sqrt{N}$ 的稳定冗余。而这一点可以被推广成稳定冗余与临时冗余的乘积下界。

5 空间下界

为便于理解，下面两个证明使用同一组内存模型假设。数组元素是任意 word-size 值，因而不能默认进一步压缩；每个数据块都必须由一个指针标识，一个指针占 $\Theta(1)$ words。索引块与长度、计数等辅助信息也计入空间，而且只会消耗更多冗余，不会使可用数据块数量增加。最后，新块刚申请时空；在元素复制完成以前，旧元素仍必须存在于原数据块中。Theorem 4.2 还使用 Definition 2.1 的约定： $s(N)$ 和 $t(N)$ 都是非递减函数。

5.1 Theorem 4.1: $\sqrt{N}$ 下界

定理。 即使操作序列只包含 **Grow** 和访问，任何可变长数组实现也必定在某些时刻使用 $N + \Omega(\sqrt{N})$ 空间，其中这里的 N 指该时刻数组的长度。

证明。 从空数组开始执行一段共 N 次的 **Grow** 操作。执行完毕时， N 个元素分布在 k 个连续数据块中。根据 k 的大小分两种情况。

若 $k \geq \sqrt{N}$ ，保存数据本身至少需要 N words，而标识这 k 个数据块至少需要 k 个指针。因此最终时刻的空间至少为

$$N + k \geq N + \sqrt{N}.$$

若 $k < \sqrt{N}$ ，由平均值原理，至少存在一个容量为 ℓ 的数据块满足

$$\ell \geq N/k > \sqrt{N}.$$

设这个大块在历史上数组长度为 $N' \leq N$ 时被申请。新块刚申请时还没有替代旧表示，因此当时已有的 N' 个元素仍在其他数据块中，新块的 ℓ 个槽必须与它们短暂共存。又因为 $N' \leq N$ ，所以

$$\ell > \sqrt{N} \geq \sqrt{N'}.$$

故该历史时刻的总空间至少为

$$N' + \ell > N' + \sqrt{N'}.$$

在定理陈述中把这个发生峰值时的数组长度 N' 重新记为 N ，便得到“某些时刻使用 $N + \Omega(\sqrt{N})$ 空间”的结论。证毕。

证明中的两个 case 可以概括为：块多会增加指针开销；块少则迫使某个块很大，而这个大块被申请时必须与旧元素共存。需要注意，定理没有声称每个稳定状态都有 $\sqrt{N}$ 冗余；第二种情况的峰值可能发生在早于最终第 N 次 **Grow** 的历史时刻。该下界也不依赖 **Grow** 究竟花费多少时间。

5.2 Theorem 4.2: 乘积下界

定理。 任何 $(s(N), t(N))$ -implementation 都必须满足

$$s(N)t(N) \geq N$$

即使只支持 **Grow** 和访问。

证明。 在第 N 次 **Grow** 完成后的稳定状态，额外空间至多为 $s(N)$ 。由于每个数据块至少需要一个指针，并且每个指针占一个常数数量的 words，数据块数至多为 $s(N)$ （若精确考虑指针表示常数，则结论只相差常数因子）。索引块的空槽和其他辅助信息同样占用 $s(N)$ ，只会让实际可保存的指针数更少，不会破坏这一上界。

N 个元素分布在至多 $s(N)$ 个数据块中。由平均值原理，至少有一个数据块 B 的容量满足

$$|B| \geq N/s(N).$$

设 B 在历史上数组长度为 $N' \leq N$ 时被申请。刚申请时 B 为空，复制完成前原有元素仍在其他数据块中，所以这一时刻的临时额外空间至少为 $|B|$ ，即

$$t(N') \geq |B| \geq N/s(N).$$

现在才使用 t 的非递减性。由于 $N' \leq N$ ，

$$t(N) \geq t(N') \geq N/s(N)$$

两边乘以 $s(N)$ ，得到

$$s(N)t(N) \geq N.$$

证毕。

这个证明中，块数上界来自指针空间，最大块下界来自平均值原理，而单调性只用于把历史时刻的 $t(N')$ 转换成最终参数 $t(N)$ 。它证明的是稳定冗余与扩缩容临时冗余之间的空间权衡，并没有推出任何 $\Omega(r)$ 更新时间下界。

Corollary 4.3。 若

$$s(N) = O(N^{1/r})$$

则 Theorem 4.2 给出

$$\begin{aligned} t(N) &= \Omega(N/s(N)) \\ &= \Omega(N^{1-1/r}). \end{aligned}$$

两个最重要的具体代入为：

r	稳定冗余 $s(N)$	必需的临时冗余下界 $t(N)$
2	$O(N^{1/2})$	$\Omega(N^{1/2})$
3	$O(N^{1/3})$	$\Omega(N^{2/3})$

$r = 2$ 对应 HAT 和 Brodник 等旧结构所处的平衡点； $r = 3$ 则预告 Section 5 的新构造。如果希望稳定状态只有立方根级冗余，某些 **Grow** 中就不可避免地出现三分之二次幂级临时冗余。

6 与论文后续内容的连接

Section 5 的 $r = 3$ 结构正好达到 $N + O(N^{1/3})$ 稳定空间和 $N + O(N^{2/3})$ 临时空间；Section 6 先得到 $N + O(rN^{1/r})$ 稳定空间，Section 7 再对 $r(N) \leq (1/2) \log N / \log \log N$ 给出 $N + O(N^{1/r})$ 稳定空间。因此 Theorem 4.2 不只是一个独立的限制，它预先给出了后续构造应该追求的指数。对固定整数 r ，上述增长条件自然成立。

然而，空间下界并不说明 **Grow/Shrink** 的摊还时间必须为 $\Omega(r)$ 。该时间下界需要 Sections 8-9 的增长博弈，并且严格适用于作者定义的标准实现。最终 review 应把“空间最优”和“时间最优”分开证明，再在结论处合并，不能只引用 Theorem 4.2 就声称整篇论文已经在所有维度最优。

7 相关工作与发表后发展

前文为了验证复杂度，已经技术性地介绍了几何扩缩容、HAT 和 Brodник 结构。本节不重复其完整操作流程，而是回答三个不同问题：这些结构为何被提出，它们在更大的动态数据结构研究中处于什么位置，以及 Tarjan-Zwick 发表后出现了哪些实现和经验工作。

7.1 可变长数组的直接前驱

标准意义下的几何扩缩容确立了最常见的基线：连续存储带来最简单的常数访问；固定增长因子带来常数摊还更新；代价是稳定状态与重建过程都可能出现 $\Theta(N)$ 冗余。减小增长因子只能改变常数比例，不能得到 $N + o(N)$ 空间。

Sitarski 1996 年提出 HAT 的直接动机来自数据库程序：查询结果长度无法预知，商业可变长数组类在扩展时执行大量复制。其原文不仅给出 Top（索引块）和 Leaves（数据块）的两

级结构，还分析了位级寻址、内存碎片和累计复制量。对于 $N = 4^m$ 个顺序加入的元素，重建复制量形成 $1 + 4 + \dots + N = (4N - 1)/3$ ，因此总复制为 $O(N)$ 。原文还估计最坏冗余约为 $2\sqrt{N}$ ，并在 1990 年代硬件上比较 HAT 与普通 C++ 数组。由此可见，HAT 从一开始就同时关注渐近空间、复制次数和工程可用性；但其旧实验只能作为历史证据，不能直接预测现代机器性能。

Brodnik 等人 1999 年从随机队列 (randomized queue) 出发，把单端可变长数组放入更完整的动态分配模型中。原文把内存块头部 (headers) 计入空间，定义 *Allocate/Deallocate/Reallocate*，并用最大块大小 $f(N)$ 与块数量 $g(N)$ 的关系

$$f(N)g(N) \geq N$$

得到 $\Omega(\sqrt{N})$ 峰值冗余下界。其最终结构使用虚拟超级块和位运算定位，并给出可执行的 *Grow*、*Shrink* 与 *Locate* 算法；通过后台复制索引指针得到最坏 $O(1)$ 操作，还给出适配伙伴系统 (buddy system) 的版本。该结构进一步导出栈、队列、随机队列、优先队列和双端队列等结果。

两项旧工作共同优化的是“任意时刻总空间”，其自然平衡点为 $N + \Theta(\sqrt{N})$ 。Tarjan-Zwick 并未否定这一旧下界，而是改变了度量：把稳定存储空间和扩缩容临时空间分开，再研究两者与更新时间的完整权衡。

7.2 简洁动态数据结构背景

在简洁动态数据结构层面，可变长数组的低冗余表示属于简洁数据结构 (succinct data structures) 的广泛背景。Raman、Raman 与 Rao 研究动态简洁结构，Raman 与 Rao 进一步研究动态字典和树，Munro 与 Rao 的综述则系统整理了相关表示方法。这个方向通常要求数据结构占用接近信息论最低编码长度的比特数，并在低阶附加空间内支持查询和更新。

Tarjan-Zwick 与该方向共享“主数据加低阶冗余”的目标，但空间基线不同：本文把 N 个任意机器字元素本身所需的 N 个机器字作为不可压缩主数据，再优化其上的指针、空槽和重组空间。因此，正文可以把它放入简洁结构研究背景，却不应把 $N + o(N)$ 个机器字与经典的比特级简洁编码当作完全相同的结论。

7.3 更强的一般动态序列问题

本文只允许在最大下标一端执行 *Grow/Shrink*。Dietz 的 list indexing 和 Fredman-Saks 的动态数据结构下界研究的是更强的秩维护问题：若在任意位置插入或删除，后续元素的逻辑下标都会改变。Tarjan-Zwick 引言据此指出，所有操作同时支持时会出现 $\Theta(\log N / \log \log N)$ 量级的界，这与本文的末端常数访问问题不是同一操作模型。

Goodrich 与 Kloss 的分层向量 (tiered vectors) 用常数层分级结构支持一般位置更新：在固定层数下可以保持常数访问，并以 N 的分数次幂时间完成插入和删除。Bille 等人 2017 年进一步实现多层分层向量，支持 $O(1)$ 访问、 $O(N^\epsilon)$ 插删和 $o(N)$ 冗余，并给出大规模 C++ 实验。这些工作说明“多层分块”也能服务于一般动态序列，但其更新时间不能与只在末端更新的 $O(r)$ 摊还界直接比较。

7.4 实践研究与发表后发展

Joannou 与 Raman 对可扩展数组 (extensible arrays) 进行了经验比较, Katajainen 则研究如何在实践中实现具有最坏情况保证的动态数组。这些研究提醒我们: 渐近冗余、访问指令数量、缓存局部性、分配器行为和实现常数是不同评价维度。Sitarski 的早期基准和 Brodnik 的伙伴系统版本也属于这条实践脉络, 而不是 Tarjan-Zwick 新下界的一部分。

Tarjan-Zwick 的初步版本发表于 SOSA 2023, 最终版本发表于 2024 年 *SIAM Journal on Computing*。最终期刊版致谢 Christian Kjær 和 Victor Brevig, 并说明他们的实现促使作者简化 Section 6.3 的常数时间访问方法。两人的 DTU 学位论文实现并测试了 Tarjan-Zwick 的结构, 补充了原文没有完全展开的重建细节; 其实验认为, 理论 $O(1)$ 定位在实际可用的较小 r 上可能慢于简单的 $O(r)$ 循环定位, 并比较了稳定冗余、临时峰值、访问和更新成本。

根据尽两年的学术研究, 2025 年出现了独立 Rust 实现 `tzarrays`, 说明该结构已经有论文作者之外的工程实现。不过, 学位论文和代码仓库不等同于新的同行评审理论结果。截至 2026-07-16 的本轮公开检索, 确认的发表后发展主要是最终期刊版本、实现与经验评估; 尚未确认有同行评审理论工作进一步改进其渐近权衡。这是带检索范围和日期的结论, 不能改写成“此后绝对没有研究”。

7.5 本文在研究脉络中的位置

相关工作可以沿三条轴概括。几何扩缩容、HAT 和 Brodnik 逐步降低可变长数组的空间浪费; 简洁数据结构提供“主数据加低阶冗余”的一般研究背景; 分层向量等结构处理更强的任意位置动态序列。Tarjan-Zwick 的独特位置, 是在末端更新模型中首次系统分离稳定存储空间与扩缩容临时空间, 并同时匹配空间和更新时间下界。

因此, A 部分最终应停在一个明确问题上: 当 $r = 3$ 时, Corollary 4.3 已经说明 $O(N^{1/3})$ 稳定冗余必然伴随 $\Omega(N^{2/3})$ 临时冗余; 接下来需要由 B 说明, Section 5 如何用大块和小块 (large/small blocks) 构造一个恰好达到这两个指数、同时保持最坏 $O(1)$ 访问和摊还 $O(1)$ 更新的结构。